

MEEA* Code Review

2022111931
이정진

Table of Contents

MEEA* Recap

- A*
- MCTS
- Path Consistency
- Flow

policyNet

- RolloutPolicyNet
- MLPModel

valueEnsemble

- ValueEnsemble
- Trainer

GraphEncoder

- GraphLayer
- GraphModel

MEEA

- Node
- MCTS_A

Experiment

- MEEA*-PC
- MEEA*-PC-NPs

1. MEEA* Recap

A* Algorithm

$$f(x) = g(x) + h(x)$$

- g : 지금까지의 누적 반응 비용
- h : cost estimate function이 예측한 남은 비용 (*heuristic*)

1. MEEA* Recap

Monte Carlo Tree Search

$$a^* = \operatorname{argmax}_{a \in \mathcal{A}} \left\{ -Q(s, a) + c_{puct} \frac{1}{|\mathcal{A}|} \frac{\sqrt{\sum_{b \in \mathcal{A}} N(s, b)}}{1 + N(s, a)} \right\}$$

- $Q(s, a)$: f -value의 경험적 추정치 (**exploitation**)
- c_{puct} : 탐험(**exploration**)의 강도를 조정하는 hyperparameter
- $\mathcal{A}$: 취할 수 있는 action 집합 (**uniform distribution**)
- $N(s, a)$: 상태 s 일 때, a 라는 행동을 선택한 횟수
- $\sum_{b \in \mathcal{A}} N(s, b)$: 특정 상태에서 가능한 모든 행동들이 선택된 횟수

1. MEEA* Recap

Path Consistency

$$\mathcal{L}(\theta) = \mathcal{L}_{RL}(\theta) + \lambda \mathcal{L}_{PC}(\theta)$$

$$\mathcal{L}_{RL}(S) = (h(s) - c(s))^2$$

$$\mathcal{L}_{PC} = (f(s) - \bar{f}(s))^2, \bar{f}(s) = \frac{1}{N+1} \sum_{i=0}^N f(s_i)$$

L_{RL} : cost estimator와 single-step model 간의 loss

L_{PC} : f -value의 consistency loss

λ : 두 loss 간의 가중치 parameter

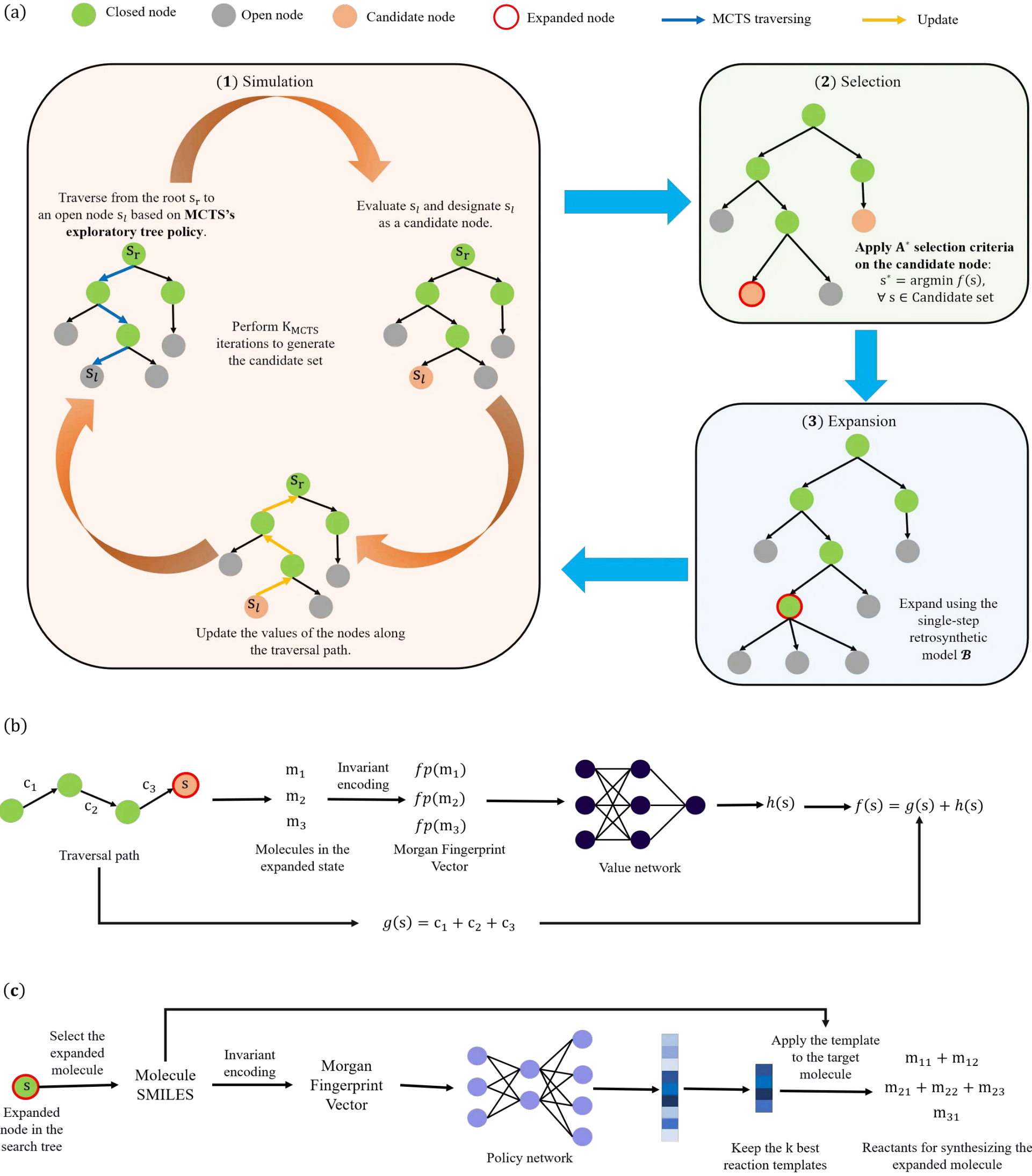
1. MEEA* Recap

Flow

1. **Single-step model**을 통해 target molecule의 **전구체들의 집합**을 찾는다.
2. **Policy network**를 통해 **Reaction cost(g-value)**를 계산한다.
3. 모든 전구체 집합들에 대해 **Value Estimation**을 진행한다.
 - a. 각 전구체 분자를 Fingerprint 형태의 **고정 vector로 변환**한다.
 - b. **fingerprint**를 **인코딩**하고 유효한 분자 feature만 집계한다.
 - c. **linear head**를 통해 **h value**를 **예측**한다.
4. **$f = g + h$ 평가 및 탐색**을 진행한다.
 - a. 각 후보 상태에 대해 f-value를 계산한다.
 - b. **pUCT**에 따라 확장할 노드를 선택한다.
5. 가장 낮은 **f value**를 갖는 노드를 우선 확장한다.

1. MEEA* Recap

Flow



2. policyNet

RolloutPolicyNet

```
1  class RolloutPolicyNet(nn.Module):
2
3      def __init__(self, n_rules, fp_dim=2048, dim=512,
4                    dropout_rate=0.3):
5
6          # 2층 MLP: fingerprint -> 은닉층 -> 템플릿 로짓
7          self.fc1 = nn.Linear(fp_dim, dim) # 첫 번째 선형층
8          self.bn1 = nn.BatchNorm1d(dim) # 배치 정규화 (학습 안정화)
9          self.dropout1 = nn.Dropout(dropout_rate) # 과적합 방지
10         self.fc3 = nn.Linear(dim, n_rules) # 출력층 (템플릿 분류)
11
12     def forward(self, x, y=None, loss_fn=nn.CrossEntropyLoss()):
13
14         # fingerprint -> 은닉층 -> BN -> 드롭아웃 -> 출력층
15         x = self.fc3(self.dropout1(self.bn1(self.fc1(x))))
16
17         if y is not None:
18             # 학습 모드: 손실 반환
19             return loss_fn(x, y)
20         else:
21             # 추론 모드: 로짓 반환
22             return x
```

- Single-Step retrosynthesis의 policy network
- Multi-Layer Perceptron
- fingerprint → hidden layer (2 layer) → Batch Norm → Dropout → output
- Cost Estimator보다 무거운 모델

2. policyNet

MLPModel

```
1  def run(self, x, topk=10):
2
3      # 1. Fingerprint 생성 및 전처리
4      arr = preprocess(x, self.fp_dim)
5
6      # 2. 신경망으로 템플릿 확률 예측
7      preds = self.net(arr)
8      preds = F.softmax(preds, dim=1) # 확률로 변환
9
10     # 3. Top-k 템플릿 선택
11     probs, idx = torch.topk(preds, k=topk)
12     rule_k = [self.idx2rules[id] for id in idx[0].numpy().tolist()]
13
14     # 4. 각 템플릿을 분자에 적용
15     reactants = []
16     scores = []
17     templates = []
18     for i, rule in enumerate(rule_k):
19         try:
20             out1 = rdchiralRunText(rule, x)
21
22             for reactant in out1:
23                 reactants.append(reactant)
24                 scores.append(probs[0][i].item()/len(out1)) # 확률을 균등 분배
25                 templates.append(rule)
26
27     return {'reactants': reactants,
28           'scores': scores,
29           'template': templates}
```

- RolloutPolicyNet의 Wrapper class
- fingerprint data 전처리 (Morgan Fingerprint, 키랄성을 고려)
- 가능도가 높은 K개의 가능한 전구체들의 집합 추출
- Softmax를 통해 probability 추출
- 확률 합이 1이 되도록 정규화 → g-value에서 사용할 probability 산출

3. valueEnsemble

ValueEnsemble

```
1  class ValueEnsemble(nn.Module):
2
3      def __init__(self, fp_dim, latent_dim, dropout_rate):
4
5          # 그래프 신경망으로 분자 집합 인코딩
6          self.graphNN = GraphModel(input_dim=fp_dim, feature_dim=latent_dim,
7                                     hidden_dim=latent_dim)
8
9          # 선형층으로 가치 예측 (바이어스 없음)
10         self.layers = nn.Linear(latent_dim, 1, bias=False)
11
12     def forward(self, fps, mask):
13
14         x = self.graphNN(fps, mask) # 분자 집합 인코딩
15         x = self.layers(x) # 가치 예측
16         return x
```

- Cost estimator
- GraphModel로 Encoding한 뒤 linear layer를 통해 남은 경로(h*)를 예측

3. valueEnsemble

Trainer (def _pass)

```
1 # === Fitting Loss 계산 ===
2
3 # 분자 개수에 따른 가중치 (적은 분자 = 높은 가중치)
4 weight = torch.FloatTensor(np.array([1 / self.num_mols] *
5                                     len(values)).astype(np.float32)).to(device)
6
7 v_pred = self.model(fps, masks)
8 fitting_loss = self.fitting_criterion(v_pred.reshape(-1), values)
9 fitting_loss = (weight * fitting_loss).mean() # 가중 평균
```

```
1 # ValueEnsemble model을 통해 예측한 남은 cost
2 r_values = self.model(reactant_fps, reactant_masks)
3
4 # Bellman 잔차: V(target) - V(reactants) - cost
5 # 이상적으로는 0이어야 함. 여기에 7을 더해 양수로 만들고 hinge loss 적용
6 r_gap = - r_values - reaction_costs + target_values + 7.
7 r_gap = torch.clamp(r_gap, min=0) # Hinge loss
8 consistency_loss = (r_gap ** 2).mean()
```

- Fitting loss(cost estimator와 policy network) 간의 차이 계산 (MSE Loss)
- 논문 코드에는 학습을 위한 'train_fitting.pkl'이 존재하지 않음
- Consistency loss 간의 Bellman residual
- A*의 Admissible condition / gradient vanishing 방지를 위해 예측값에 -7
- 논문 코드에는 학습을 위한 'train_fitting.pkl'이 존재하지 않음

3. valueEnsemble

Trainer (def_pass)

```
1 # === 전체 손실 및 역전파 ===  
2 loss = fitting_loss + consistency_loss  
3 loss.backward()  
4 self.optim.step() #Adam
```

- fitting loss와 consistency loss의 가중치를 동일하게 적용

4. GraphEncoder

GraphLayer

```
1  class GraphLayer(nn.Module):
2
3      def __init__(self, in_features, out_features):
4
5          super(GraphLayer, self).__init__()
6          self.linear = nn.Linear(in_features, out_features) # 선형 변환
7          self.drop = nn.Dropout(0.1) # 과적합 방지를 위한 드롭아웃 (10%)
8
9      def forward(self, x):
10
11          x = self.drop(F.relu(self.linear(x)))
12          return x
```

- end-to-end representation learning
- fingerprint data를 학습에 용이하게 변형
- Linear transformation, ReLU, Dropout

4. GraphEncoder

GraphModel

```
1  class GraphModel(nn.Module):
2
3      def __init__(self, input_dim, feature_dim, hidden_dim):
4
5          super(GraphModel, self).__init__()
6          self.hidden_dim = hidden_dim
7          self.graphlayer = GraphLayer(input_dim, hidden_dim) # 단일 그래프
8
9      def forward(self, x, mask):
10
11          x = self.graphlayer(x) # (batch_size, num_molecules,
12                                # hidden_dim)
13
14          # 마스크를 hidden_dim만큼 복제하여 각 특징에 적용
15          mask = mask[:, :, None].repeat(1, 1, self.hidden_dim)
16
17          # 마스킹 후 합산 (유효한 분자들만 집계)
18          x = torch.sum(x * mask, dim=1) # (batch_size, hidden_dim)
19          return x
```

- Masking → padding slot을 제외, 유효 분자들의 임베딩을 합산

5. MEEA

Node

$$a^* = \operatorname{argmax}_{a \in \mathcal{A}} \left\{ -Q(s, a) + c_{puct} \frac{1}{|\mathcal{A}|} \frac{\sqrt{\sum_{b \in \mathcal{A}} N(s, b)}}{1 + N(s, a)} \right\}$$

```
1 def select_child(self, min_max_stats):
2
3     #pUCT 기반 child node 선택
4     action_score = self.child_Q(min_max_stats) + self.child_U() -
5     self.child_illegal
6     best_move = np.argmax(action_score)
7     return best_move
```

```
1 def prepare_expand(model_path, gpu=-1):
2
3     # 템플릿 기반 역합성 예측 모델 로드
4     one_step = MLPModel(model_path, './saved_model/
5     template_rules.dat', device=device)
6     return one_step
```

```
1 def prepare_value(model_f, gpu=None):
2
3     # 입력 차원: 2048, 은닉층 차원: 128, 드롭아웃: 0.1
4     model = ValueEnsemble(2048, 128, 0.1).to(device)
5     model.load_state_dict(torch.load(model_f, map_location=device))
6     model.eval() # 평가 모드로 설정
7     return model
```

- pUCT tree policy를 바탕으로 확장할 child node의 index 반환
- child_Q(1 - cost), child_U가 뒤의 항
- policy Network (MLPModel) 호출
- cost Estimator (ValueEnsemble) 호출

5. MEEA

MCTS_A (select_a_leaf, select)

```
1 def select_a_leaf(self):
2
3     current = self.root
4     while True:
5         current.visited_time += 1 # 방문 횟수 증가
6         if not current.is_expanded: # 확장되지 않은 노드면 반환
7             return current
8
9     # PUCT로 최선의 자식 선택하여 계속 탐색
10    best_move = current.select_child(self.min_max_stats)
11    current = current.children[best_move]
```

```
1 def select(self):
2
3     # opening_size만큼 리프 노드를 샘플링
4     openings = [self.select_a_leaf() for _ in range(self.opening_size)]
5
6     # 그 중 f 값(비용)이 가장 낮은 노드 선택
7     stats = [opening.f for opening in openings]
8     index = np.argmin(stats)
9     return openings[index]
```

- pUCT policy를 통해 최선의 자식 노드 중 리프 노드 선택
- 선택된 노드들은 candidate set이 됨
- 방문 시 N+1
- f value가 가장 작은 실제로 확장할 노드 선택

5. MEEA

MCTS_A (expand)

```
1  for i in range(len(expanded_policy['scores'])):
2      reactant = ...
3
4      # 반응 비용 계산 (음의 로그 확률)
5      cost = -np.log(expanded_policy['scores'][i])
6
7      # 모든 반응물이 알려진 물질인 경우 (성공!)
8      if len(reactant) == 0:
9          child = Node([], 0, cost=cost, prior=priors[i], ...)
10         return True, child # 성공 경로 도달
11     else:
12         # 아직 분해가 필요한 분자가 있는 경우
13         h = value_fn(self.value_model, reactant, self.device) # 휴리스
14         # 틱 값 계산
15         child = Node(reactant, h, cost=cost, prior=priors[i], ...)
16
17         # 이미 방문한 상태인지 확인 (사이클 방지)
18         if ''.join(reactant) in self.visited_state:
19             node.child_illegal[child.fmove] = 1000 # 불법 상태 표시
```

- Single step retrosynthesis를 실제로 수행
- 방문한 정책과 Score는 저장

5. MEEA

MCTS_A (update)

```
1 def update(self, node):
2     stat = node.f # 현재 노드의 f 값
3     self.min_max_stats.update(stat) # 전역 통계 업데이트
4
5     # 루트까지 역전파하며 각 조상 노드의 경로 통계 업데이트
6     current = node
7     while current is not None:
8         current.f_mean_path.append(stat) # 이 경로의 f 값 기록
9         current = current.parent
```

- heuristic value를 부모 노드들에 backpropagate
→ Node에 있는 Q value를 업데이트
- 방문한 정책과 Score는 저장

```
1 def child_Q(self, min_max_stats):
2     child_Qs = []
3     for child in self.children:
4         if len(child.f_mean_path) == 0:
5             child_Qs.append(0.0) # 방문하지 않은 노드
6         else:
7             # 정규화된 평균 비용을 품질 점수로 변환 (낮은 비용 = 높은 품질)
8             child_Qs.append(1 -
9                 np.mean(min_max_stats.normalize(child.f_mean_path)))
9     return np.array(child_Qs)
```

Node

6. Experiment

MEEA_PC & MEEA_PC_NPs

데이터셋, 목표 분자 개수, 탐색횟수($K_{\{MCTS\}}$), cpuct, Success rate, Avg Depth, Avg Time(s), Total Time

```
(mee_a_gpu) gpu_03@linuxserver2:~/MEEA$ cat result_simulation.txt
USPTO      100      500      4.0      0.9316    8.24      933.60    00:15:33
ClinTox    100      500      4.0      0.5908   16.58     5230.49   01:27:10
logS        100      500      4.0      0.8157    9.47     1472.59   00:24:32
Toxicity_LD50 100      500      4.0      0.7305   12.61     2725.66   00:45:25
BBBP        100      500      4.0      0.6623   14.49     4278.17   01:11:18
Ames        100      500      4.0      0.7750   11.39     3195.18   00:53:15
logP        100      500      4.0      0.7279   12.26     6871.19   01:54:31
bace        100      500      4.0      0.5262   20.19     4649.17   01:17:29
DPP4        100      500      4.0      0.9419    7.45     3258.33   00:54:18
SVS         100      500      4.0      0.7167   13.55     7551.92   02:05:51
natural     100     3000      4.0      0.0103   31.70     8463.80   02:21:03
```

- Depth가 깊을 수록 정확도 감소, Avg 시간 증가
- 11 step 이상일 때 성능이 확연히 감소
- natural products의 경우 매우 깊은 depth
→ accuracy low
- 탐색 횟수를 변경해가며 테스트 진행할 필요 있음
(실제로 탐색 횟수가 적으면(많으면) MCTS(A*)적인 성격을 띄는지)